

Probabilistic Programming

Marius Popescu

popescunmarius@gmail.com

2025 - 2026

Introduction

What is Probabilistic Programming

Probabilistic programming languages aim to unify general purpose programming with probabilistic modeling; literally, users specify a probabilistic model in its entirety (e.g., by writing code that generates a sample from the joint distribution) and inference follows automatically given the specification

<http://www.probabilistic-programming.org>

Probabilistic programming is about performing Bayesian inference using the tools of computer science: programming language for model denotation and statistical inference algorithms for computing the conditional distribution of program inputs that could have given rise to the observed program output

A programming paradigm that provides a language based on random variables and stochastic control flow to construct a broad class of probabilistic models

Frequentist Vs Bayesian Approaches in Statistics

Frequentist

Assume that probability is the long-run frequency of events

The probability of plane accidents under a frequentist philosophy is interpreted as the long-term frequency of plane accidents.

How to assign probabilities to outcomes of presidential elections? The election itself only happens once!

Bayesian

Interpret a probability as measure of *belief*, or confidence, of an event occurring. Simply, a probability is a summary of an opinion. An individual who assigns a belief of 0 to an event has no confidence that the event will occur; conversely, assigning a belief of 1 implies that the individual is absolutely certain of an event occurring.

This definition agrees with the probability of a plane accident example, for having observed the frequency of plane accidents, an individual's belief should be equal to that frequency, excluding any outside information.

Similarly, under this definition of probability being equal to beliefs, it is meaningful to speak about probabilities (beliefs) of presidential election outcomes: how confident are you candidate A will win?

(Bayesian) Probabilistic Modeling

Problem: given i.i.d. data $X = (x_1, x_2 \dots, x_n)$ from distribution $p(x|\theta)$ one needs to estimate θ

Frequentist framework: use maximum likelihood estimation (MLE)

$$\theta_{ML} = \operatorname{argmax} p(X|\theta) = \operatorname{argmax} \prod_{i=1}^n p(x_i|\theta) = \operatorname{argmax} \sum_{i=1}^n \log p(x_i|\theta)$$

Bayesian framework: encode uncertainty about θ in a *prior* $p(\theta)$ and apply Bayesian inference to find *the posterior*:

$$p(\theta|X) = \frac{\prod_{i=1}^n p(x_i|\theta) p(\theta)}{\int \prod_{i=1}^n p(x_i|\theta) p(\theta) d\theta}$$

Example: Coin Tossing

- We have a coin which may be fair or not
- The task is to estimate a probability θ of landing heads up
- Data: 2 tries with a result (H,H)



Head (H)



Tail (T)

Example: Coin Tossing

- We have a coin which may be fair or not
- The task is to estimate a probability θ of landing heads up
- Data: 2 tries with a result (H,H)



Head (H)



Tail (T)

Frequentist framework:

In all experiments the coin
landed heads up
 $\theta_{ML}=1$



The coin is not fair and
always lands heads up

Example: Coin Tossing

- We have a coin which may be fair or not
- The task is to estimate a probability θ of landing heads up
- Data: 2 tries with a result (H,H)

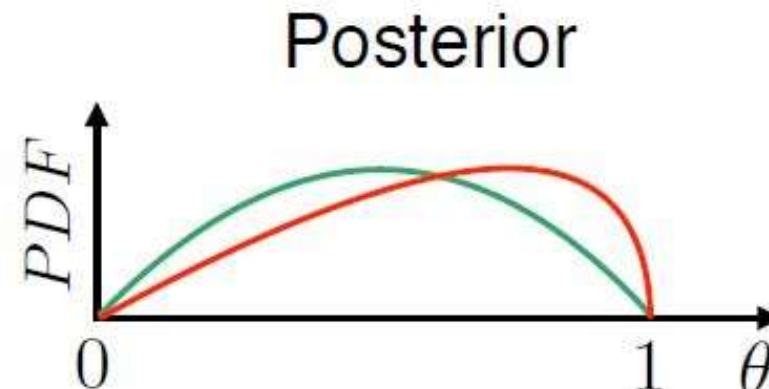
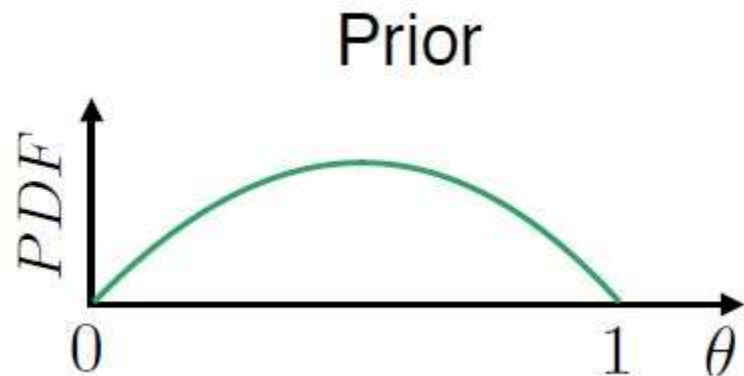


Head (H)



Tail (T)

Bayesian framework:



Example: Coin Tossing

- We have a coin which may be fair or not
- The task is to estimate a probability θ of landing heads up
- Data: 1000 tries with a result of 489 tails and 511 heads



Head (H)



Tail (T)

Both frameworks:

Sufficient amount of data
matches our expectations



The coin is fair

Bayesian Modeling: The Problem

Bayesian framework: encode uncertainty about θ in a *prior* $p(\theta)$ and apply Bayesian inference to find *the posterior*:

$$p(\theta|X) = \frac{\prod_{i=1}^n p(x_i|\theta) p(\theta)}{\int \prod_{i=1}^n p(x_i|\theta) p(\theta) d\theta}$$

May be intractable

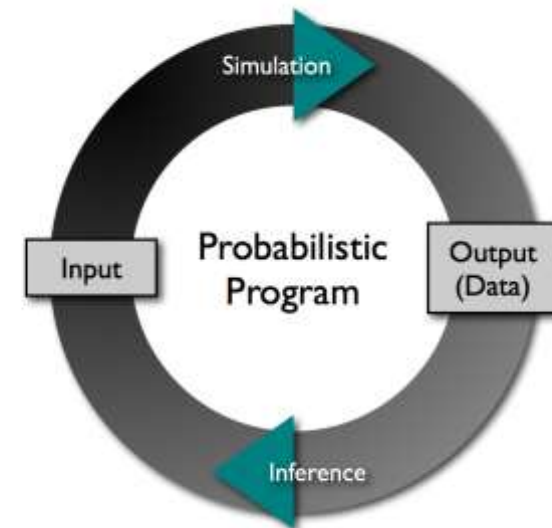
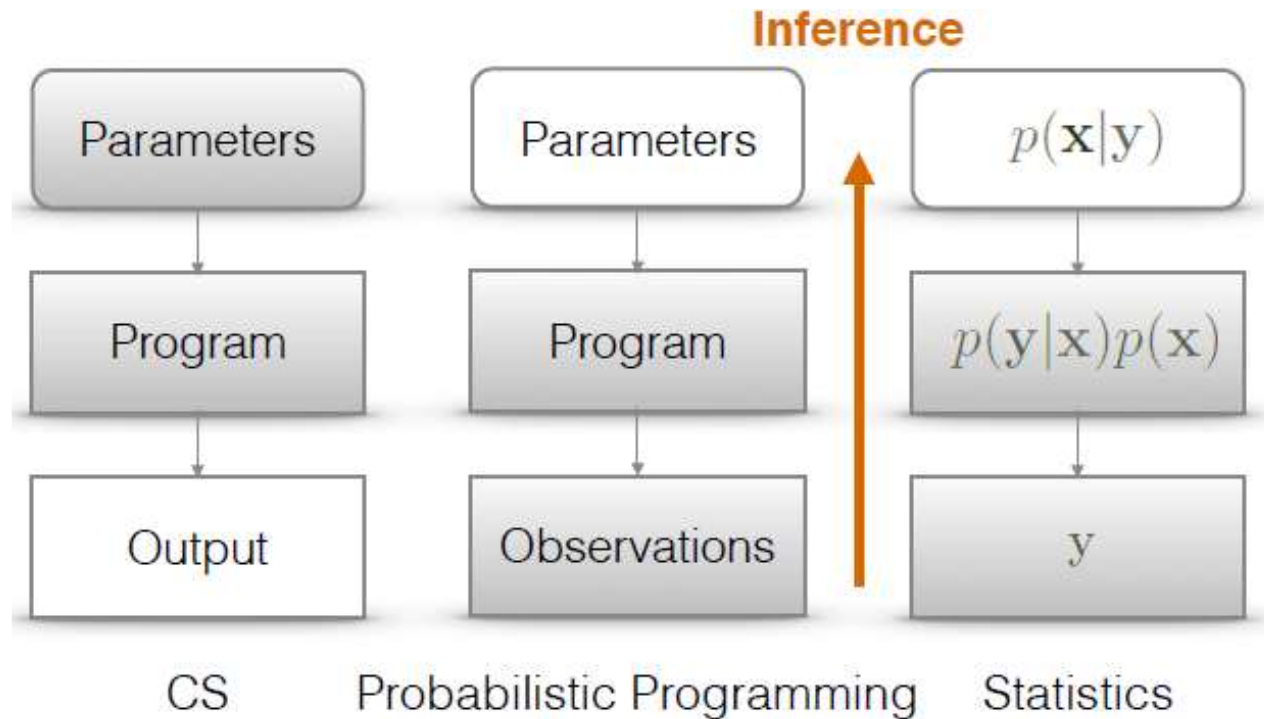
What is Probabilistic Programming

Probabilistic programming languages are to the *Bayesian or probabilistic machine learning* as automated differentiation tools are to the *deep learning*

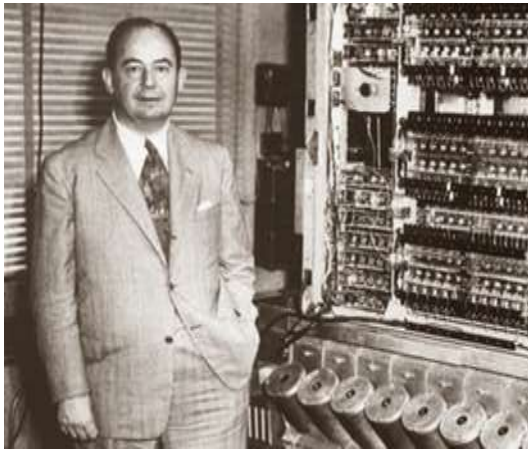
The rapid growth of deep learning has been triggered largely by the emergence of programming language tools that automate the tedious and troublesome derivation and calculation of gradients for optimization

Probabilistic programming aims to build and deliver a toolchain that does the same for probabilistic machine learning

What is Probabilistic Programming



Some History



Von Neumann, J. (1956).

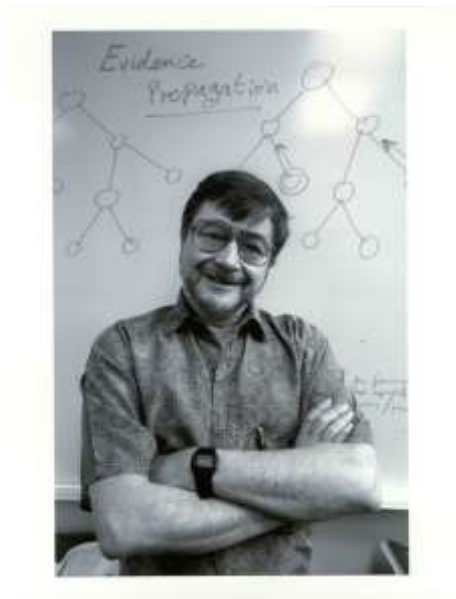
Probabilistic logics and the
synthesis of reliable organisms
from unreliable components



Shannon, C. E. (1958).

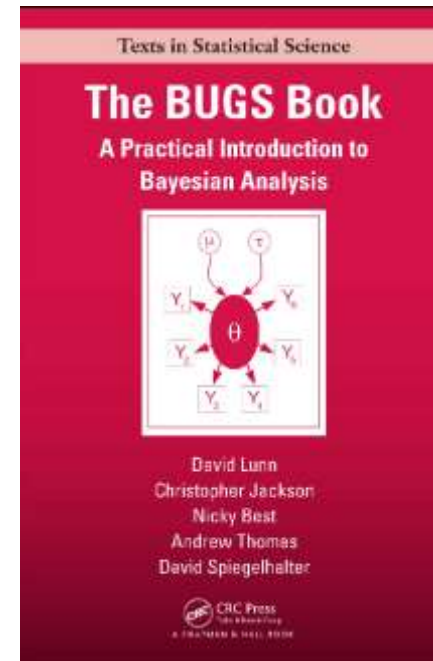
Von Neumann's contributions to automata theory.

Some History



Pearl, J. (1982).

“Reverend Bayes on inference engines:
A distributed hierarchical approach,”
Proceedings, AAAI-82



The BUGS (**B**ayesian inference **U**sing **G**ibbs **S**ampling) project is concerned with flexible software for the Bayesian analysis of complex statistical models using Markov chain Monte Carlo (MCMC) methods. The project began in 1989 in the MRC Biostatistics Unit, Cambridge.

Is it useful? Is it hot?

Why to study probabilistic programming?



nature

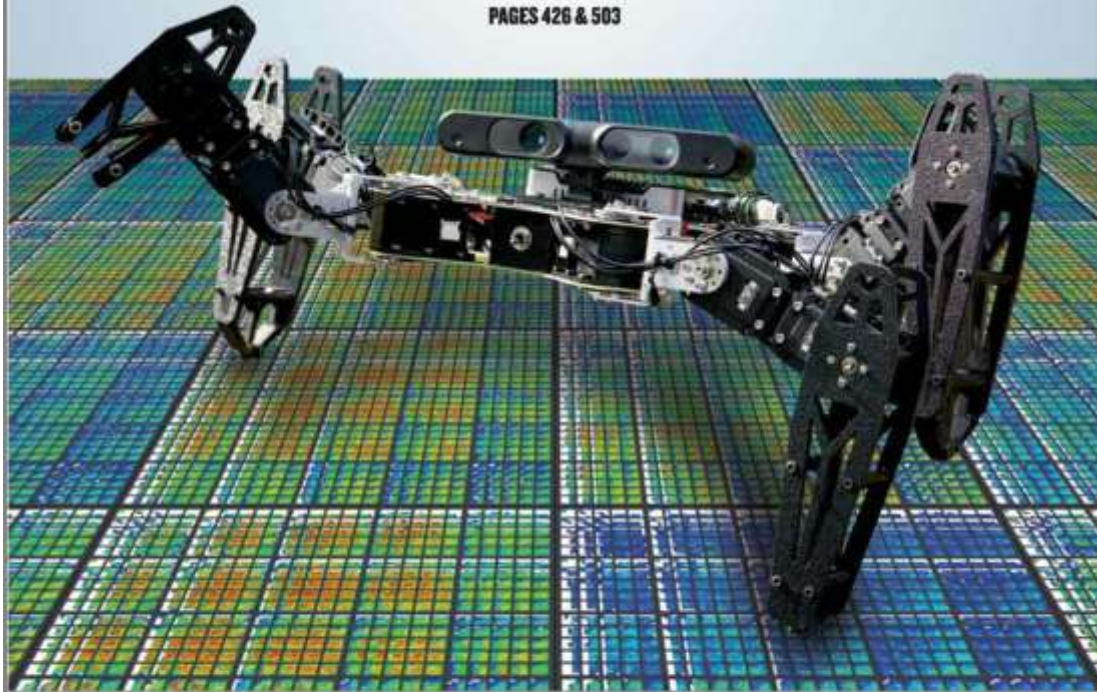
THE INTERNATIONAL WEEKLY JOURNAL OF SCIENCE

INSIGHT
Machine
Intelligence

Back on its feet

Using an intelligent trial-and-error learning algorithm this robot adapts to injury in minutes

PAGES 426 & 503



COGNITION

WHY FISH NEED TO BE CLEVER

Social behaviours need plenty of brainpower

PAGE 412

ARTIFICIAL INTELLIGENCE

LIVING WITH ROBOTS

AI researchers' ethics prescriptions

PAGE 415

HUMAN EVOLUTION

ANOTHER FACE IN THE CROWD

A new hominin from Ethiopia's middle Pliocene

PAGES 432 & 483

NATURE.COM/NATURE

28 May 2015 £10

Vol. 521, No. 7553



REVIEW

doi:10.1038/nature14539

Deep learning

Yann LeCun^{1,2}, Yoshua Bengio³ & Geoffrey Hinton^{4,5}

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

REVIEW

doi:10.1038/nature14540

Reinforcement learning improves behaviour from evaluative feedback

Michael L. Littman¹

Reinforcement learning is a branch of machine learning concerned with using experience gained through interacting with the world and evaluative feedback to improve a system's ability to make behavioural decisions. It has been called the artificial intelligence problem in a microcosm because learning algorithms must act autonomously to perform well and achieve their goals. Partly driven by the increasing availability of rich data, recent years have seen exciting advances in the theory and practice of reinforcement learning, including developments in fundamental technical areas such as generalization, planning, exploration and empirical methodology, leading to increasing applicability to real-life problems.

REVIEW

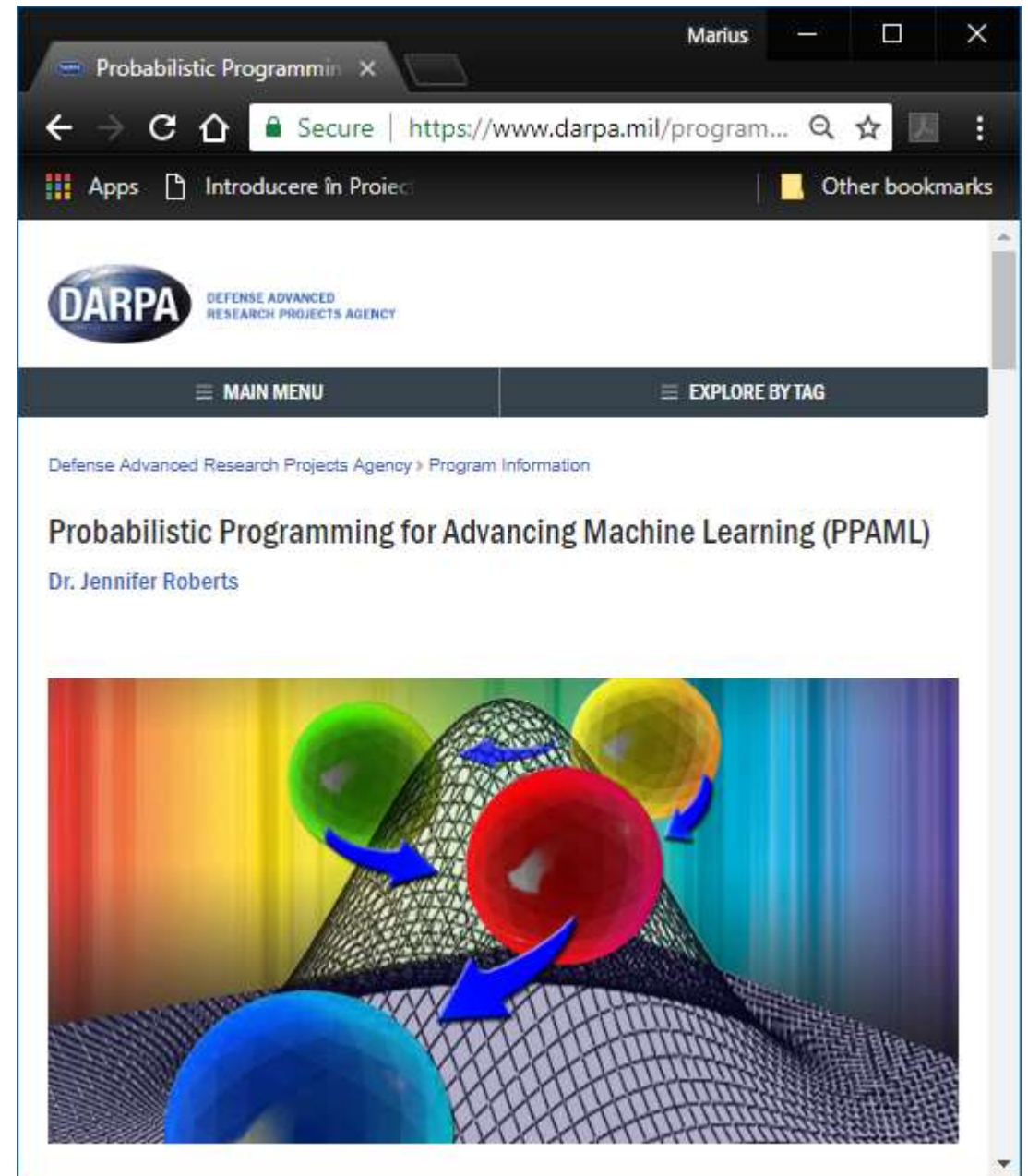
doi:10.1038/nature14541

Probabilistic machine learning and artificial intelligence

Zoubin Ghahramani¹

How can a machine learn from experience? Probabilistic modelling provides a framework for understanding what learning is, and has therefore emerged as one of the principal theoretical and practical approaches for designing machines that learn from data acquired through experience. The probabilistic framework, which describes how to represent and manipulate uncertainty about models and predictions, has a central role in scientific data analysis, machine learning, robotics, cognitive science and artificial intelligence. This Review provides an introduction to this framework, and discusses some of the state-of-the-art advances in the field, namely, probabilistic programming, Bayesian optimization, data compression and automatic model discovery.

PPAML started in March 2013 and is scheduled to run 46 months, with three phases of activity through 2017



MIT Probabilistic Computing Project

About the Lab

Latest News

Talks & Podcasts

People ▾

Publications

Industry

Classes & Resources ▾



Welcome

On this website, we describe our mission, our projects and members, the publications and resources we have produced, our curriculum in probabilistic computing, our news items, and how you can get involved.

Contact

Vikash Mansinghka

Principal Research Scientist

Dept. of Brain and Cognitive Sciences

[43 Vassar St](#)

Cambridge MA 02139

office: 46-4094A

lab: 46-5121

email: probcomp-assist@csail.mit.edu

We aim to improve our ability to engineer artificial intelligence, reverse-engineer natural intelligence, and deploy applications that increase our collective intelligence and well-being.

Our work integrates probabilistic inference, generative models, and Monte Carlo methods into the building blocks of software, hardware, and other computational systems. For example, we have developed [high-level probabilistic programming languages](#), [automated Bayesian data modeling systems](#), [Bayesian inverse graphics](#) approaches to 3D computer vision, and [near-optimal algorithms, circuits, and hardware architectures](#) for Monte Carlo. We test our results by collaborating with domain experts on practical applications.

In addition to core academic research and teaching, we mentor engineers and entrepreneurs, develop open source software, and lead hands-on AI workshops for industry. These activities have led to: VC-backed startups acquired by [Salesforce \(2012\)](#) and [Tableau \(2018\)](#), the founding in 2020 of Common Sense Machines, and a new [Intel Center for Probabilistic Computing](#). We also carry out joint development and field testing with partners from both the public and private sectors.

[Contact us](#) to get involved in testing or contributing

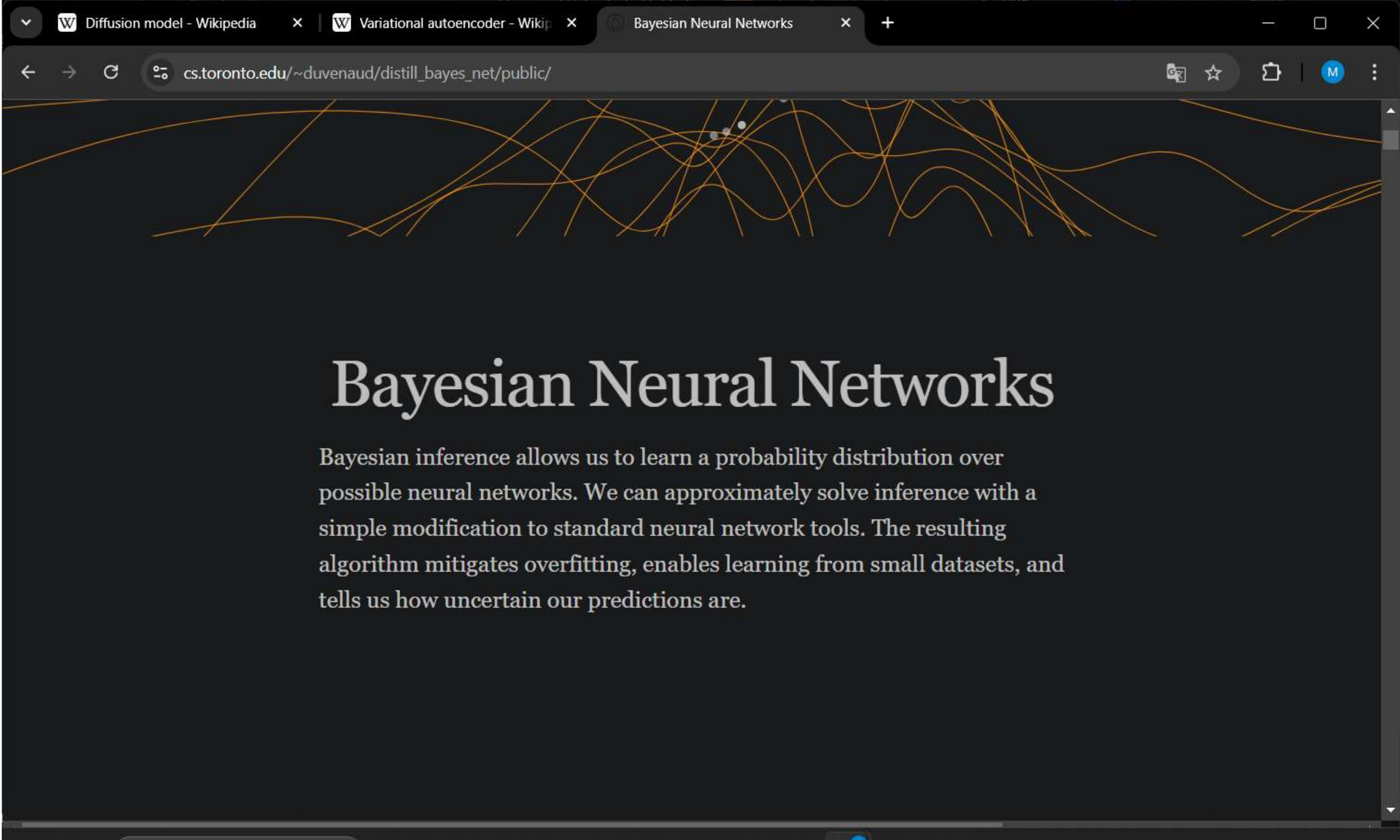
Latest News

- April 2022: Our research paper [Estimators of Entropy and Information via Inference in Probabilistic Models](#) has been presented at AISTATS 2022 and covered by MIT News.
- October 2021: Our research on [3DP3: 3D Scene Perception via Probabilistic Programming](#) has been accepted for publication at NeurIPS 2021.
- August 2021: Our research on [probabilistic programming with fast exact symbolic inference](#) was covered by MIT News





Relation with
Deep learning



Bayesian Neural Networks

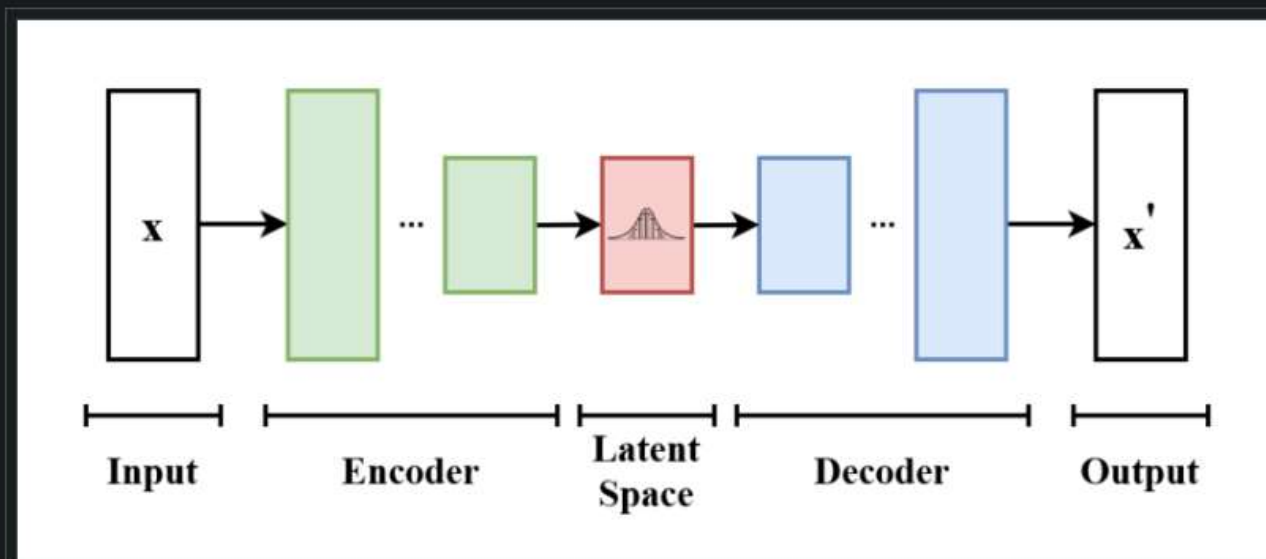
Bayesian inference allows us to learn a probability distribution over possible neural networks. We can approximately solve inference with a simple modification to standard neural network tools. The resulting algorithm mitigates overfitting, enables learning from small datasets, and tells us how uncertain our predictions are.



文 8 languages

Read Edit View history Tools ▾

In machine learning, a **variational autoencoder (VAE)** is an artificial neural network architecture introduced by Diederik P. Kingma and Max Welling.^[1] It is part of the families of probabilistic graphical models and variational Bayesian methods.^[2]



Diffusion model

 10 languages

[Article](#) [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#)

From Wikipedia, the free encyclopedia

In [machine learning](#), **diffusion models**, also known as **diffusion probabilistic models** or **score-based generative models**, are a class of [latent variable generative](#) models. A diffusion model consists of three major components: the forward process, the reverse process, and the sampling procedure.^[1] The goal of diffusion models is to learn a [diffusion process](#) for a given dataset, such that the process can generate new elements that are distributed

Part of a series on

Machine learning and data mining

Paradigms [\[show\]](#)

Problems [\[show\]](#)

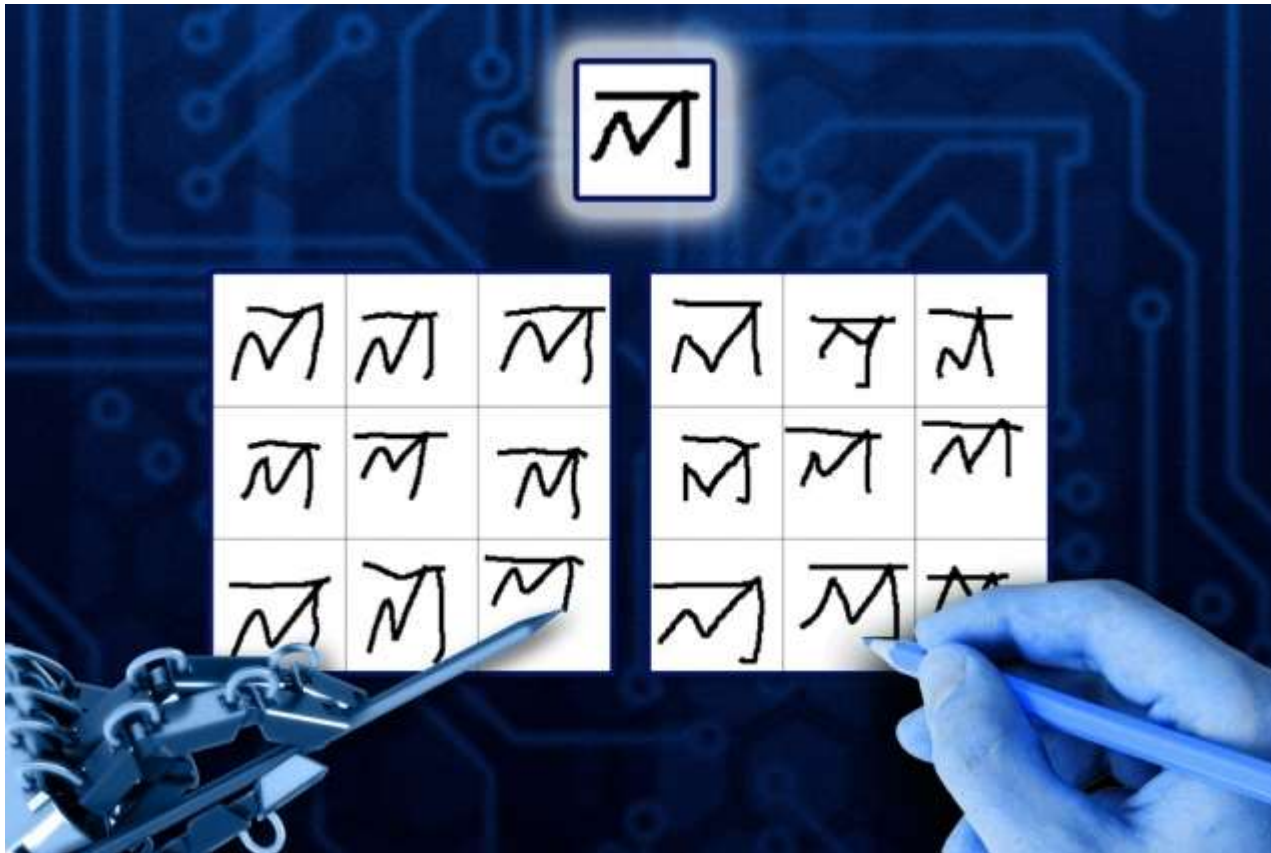
Supervised learning [\[show\]](#)
([classification](#) • [regression](#))

Clustering [\[show\]](#)



Some
Accomplishments

Humans and machines were given an image of a novel character (top) and asked to produce new versions.



RESEARCH ARTICLES

COGNITIVE SCIENCE

Human-level concept learning through probabilistic program induction

Brenden M. Lake,^{1*} Ruslan Salakhutdinov,² Joshua B. Tenenbaum³

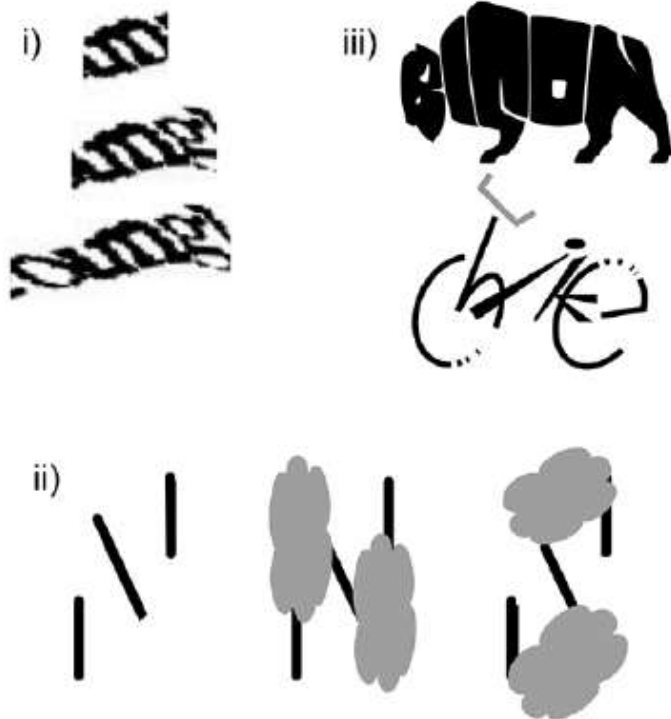
People learning new concepts can often generalize successfully from just a single example, yet machine learning algorithms typically require tens or hundreds of examples to perform with similar accuracy. People can also use learned concepts in richer ways than conventional algorithms—for action, imagination, and explanation. We present a computational model that captures these human learning abilities for a large class of simple visual concepts: handwritten characters from the world's alphabets. The model represents concepts as simple programs that best explain observed examples under a Bayesian criterion. On a challenging one-shot classification task, the model achieves human-level performance while outperforming recent deep learning approaches. We also present several “visual Turing tests” probing the model’s creative generalization abilities, which in many cases are indistinguishable from human behavior.



Common sense and context affect letter form perception: (i) m vs u and n. (ii) the same line segments are interpreted as N or S depending on occluder positions (iii) perception of the shapes aids the recognition .



c



A generative vision model that trains with high data efficiency and breaks text-based CAPTCHAs

D. George,* W. Lehrach, K. Kansky, M. Lázaro-Gredilla,* C. Laan, B. Marthi, X. Lou, Z. Meng, Y. Liu, H. Wang, A. Lavin, D. S. Phoenix

Vicarious AI, 2 Union Square, Union City, CA 94587, USA.

*Corresponding author. Email: dileep@vicarious.com (D.G.); miguel@vicarious.com (M.L.-G.)

Learning from few examples and generalizing to dramatically different situations are capabilities of human visual intelligence that are yet to be matched by leading machine learning models. By drawing inspiration from systems neuroscience, we introduce a probabilistic generative model for vision in which message-passing based inference handles recognition, segmentation and reasoning in a unified way. The model demonstrates excellent generalization and occlusion-reasoning capabilities, and outperforms deep neural networks on a challenging scene text recognition benchmark while being 300-fold more data efficient. In addition, the model fundamentally breaks the defense of modern text-based CAPTCHAs by generatively segmenting characters without CAPTCHA-specific heuristics. Our model emphasizes aspects like data efficiency and compositionality that may be important in the path toward general artificial intelligence.

Supplementary Materials for

A generative vision model that trains with high data efficiency and breaks text-based CAPTCHAs

Table S5: Classification results on noiseless MNIST data. Standard deviations are reported in parenthesis.

# training examples per class	1	5	10	20	30	50	100
Pool size	57	45	45	37	37	33	25
Perturbation factor	1.0	1.0	2.0	2.0	2.0	2.0	2.0
RCN (%)	76.59 (5.66)	92.04 (1.30)	94.99 (0.72)	96.47 (0.29)	97.28 (0.18)	97.54 (0.20)	97.89 (0.16)
VGG (%)	54.18 (4.96)	84.02 (2.20)	90.77 (0.86)	94.65 (0.56)	95.80 (0.82)	96.87 (0.43)	97.77 (0.24)
LeNet-SHREC-Mask (%)	51.02 (4.56)	79.01 (2.33)	87.47 (1.09)	90.69 (1.13)	92.64 (0.62)	94.17 (0.84)	95.47 (0.48)
LeNet (%)	46.88 (4.85)	75.15 (2.94)	86.39 (2.19)	91.56 (0.85)	94.28 (0.91)	95.38 (0.91)	96.84 (0.36)
LeNet-Contours (%)	39.80 (4.17)	65.74 (2.97)	72.00 (4.71)	75.13 (4.02)	74.28 (4.36)	76.33 (4.69)	77.98 (6.04)
CNN-Contours (%)	38.08 (3.71)	65.05 (3.28)	75.04 (1.66)	79.82 (0.78)	83.42 (1.54)	85.63 (1.41)	88.20 (0.77)
CNN-SHREC-Contours (%)	28.52 (3.21)	52.10 (3.08)	65.56 (1.73)	74.72 (1.32)	78.86 (2.51)	82.28 (1.63)	86.42 (0.81)
CPM (%)	68.86	83.79	~ 87	N/A	N/A	N/A	N/A

The Course

Course Goal & Objectives

- Understanding of the basic concepts and mechanisms of the probabilistic programming
- Ability of using a probabilistic programming framework
- Using probabilistic programming for building a probabilistic model of a phenomenon, inferring the model parameters given the model and the data, criticizing the model

Tools



PyMC: a probabilistic programming library for Python

(<https://www.pymc.io/welcome.html>)



Edward: A library for probabilistic modeling, inference, and criticism

(<http://edwardlib.org/>)



TensorFlow Probability: A library for probabilistic reasoning and statistical analysis

(<https://www.tensorflow.org/probability>)



PYRO: Deep Universal Probabilistic Programming

(<https://pyro.ai/>)



Gen: An open-source stack for generative modeling and probabilistic inference

(<https://www.gen.dev>)



[Home](#)

PyMC is a probabilistic programming library for Python that allows users to build Bayesian models with a simple Python API and fit them using Markov chain Monte Carlo (MCMC) methods.

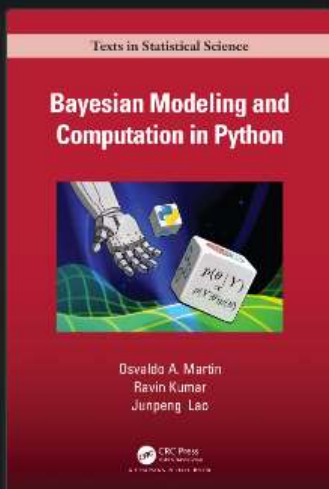
Features

PyMC strives to make Bayesian modeling as simple and painless as possible, allowing users to focus on their problem rather than the methods.

Here is what sets it apart:

- **Modern:** Includes state-of-the-art inference algorithms, including MCMC (NUTS) and variational inference (ADVI).
- **User friendly:** Write your models using friendly Python syntax. [Learn Bayesian modeling](#) from the many [example notebooks](#).
- **Fast:** Uses [PyTensor](#) as its computational backend to compile through C, Numba or JAX, [run your models on the GPU](#), and benefit from complex graph-optimizations.
- **Batteries included:** Includes probability distributions, Gaussian processes, ABC, SMC and much more. It integrates nicely with [ArviZ](#) for visualizations and diagnostics, as well as [Bambi](#) for high-level mixed-effect models.
- **Community focused:** Ask questions on [discourse](#), join [MeetUp events](#), follow us on [Twitter](#), and start [contributing](#).

Books

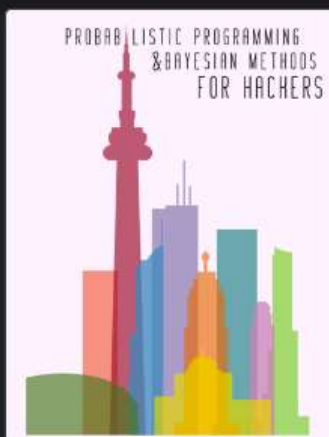


Bayesian Modeling and Computation in Python

By Osvaldo Martin, Ravin Kumar and Junpeng Lao

Hands on approach with PyMC and ArviZ focusing on the practice of applied statistics.

[Website + code](#)



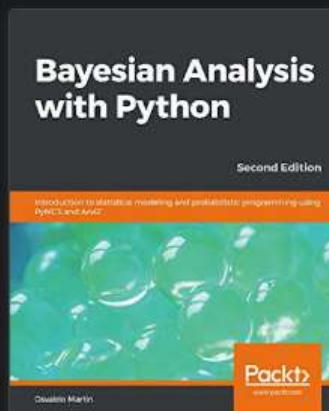
Bayesian Methods for Hackers

By Cameron Davidson-Pilon

The “hacker” in the title means learn-as-you-code. This hands-on introduction teaches intuitive definitions of the Bayesian approach to statistics, workflow and decision-making by applying them using PyMC.

[Github repo](#)

[Project homepage](#)



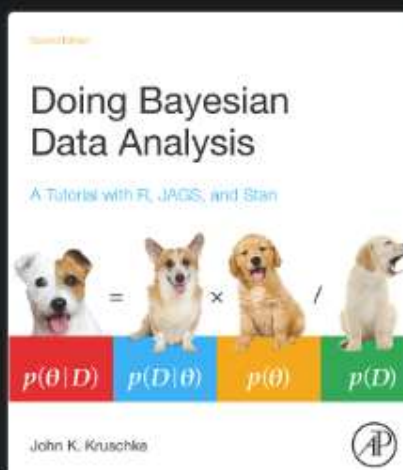
Bayesian Analysis with Python

By Osvaldo Martin

A great introductory book written by a maintainer of PyMC. It provides a hands-on introduction to the main concepts of Bayesian statistics using synthetic and real data sets. Mastering the concepts in this book is a great foundation to pursue more advanced knowledge.

[Book website](#)

[Code and errata in PyMC 3.x](#)



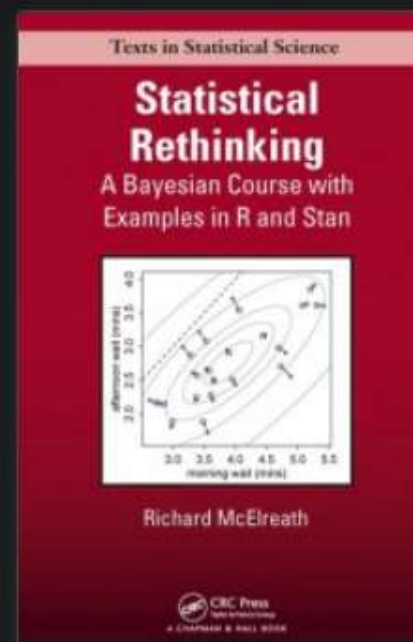
Doing Bayesian Data Analysis

By John K. Kruschke

Principled introduction to Bayesian data analysis, with practical exercises. The book’s original examples are coded in R, but notebooks with a PyMC port of the code are available through the links below.

[Book website](#)

[PyMC port of the second edition’s code](#)



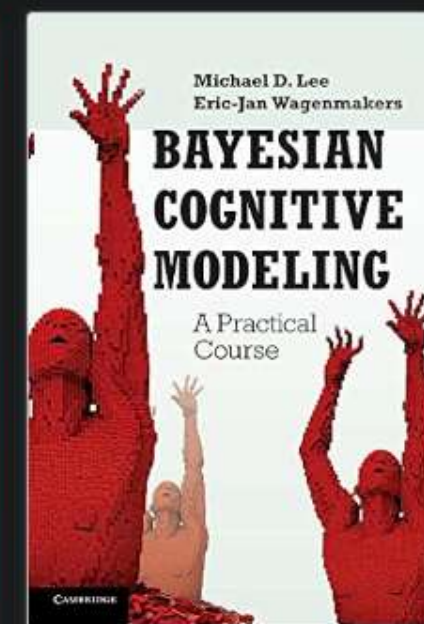
Statistical Rethinking

By Richard McElreath

A Bayesian Course with Examples in R and Stan.

[Book website](#)

[PyMC 3.x port of the code](#)



Bayesian Cognitive Modeling: A Practical Course

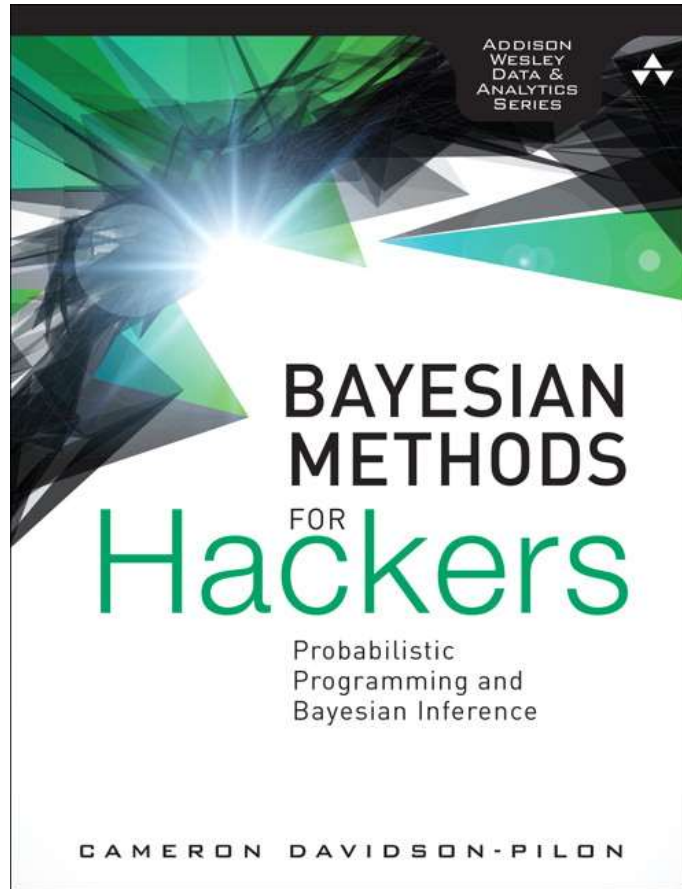
By Michael Lee and Eric-Jan Wagenmakers

Focused on using Bayesian statistics in cognitive modeling.

[Book website](#)

[PyMC 3.x implementations](#)

Bibliography



<https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>

<https://arxiv.org/abs/1809.10756>

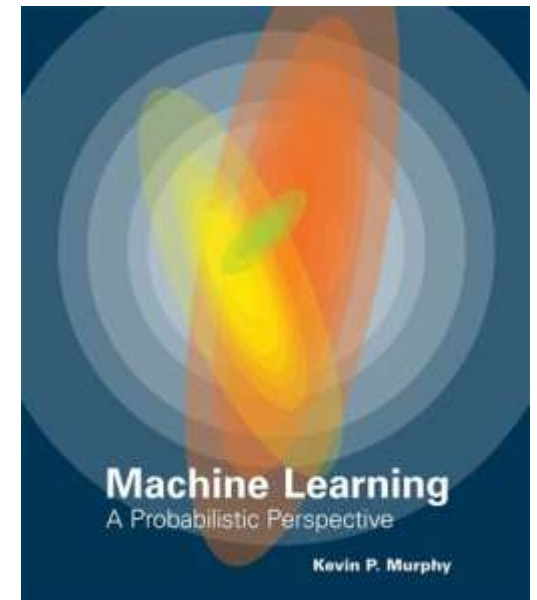
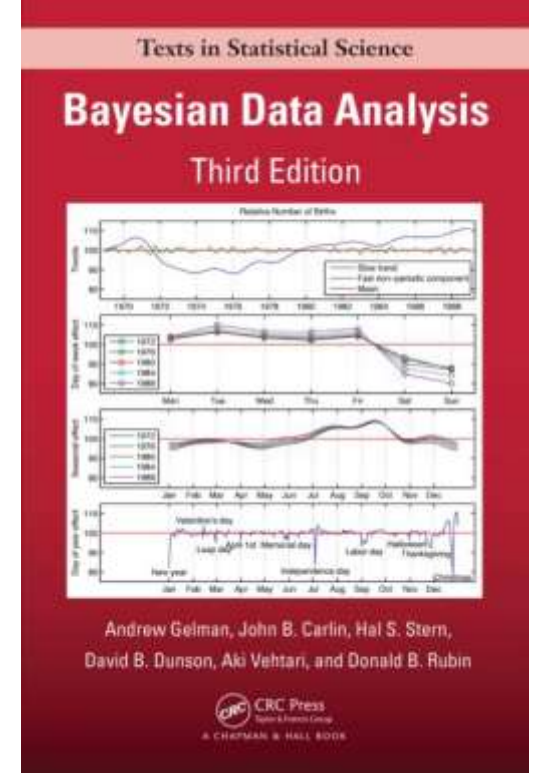
An Introduction to Probabilistic Programming

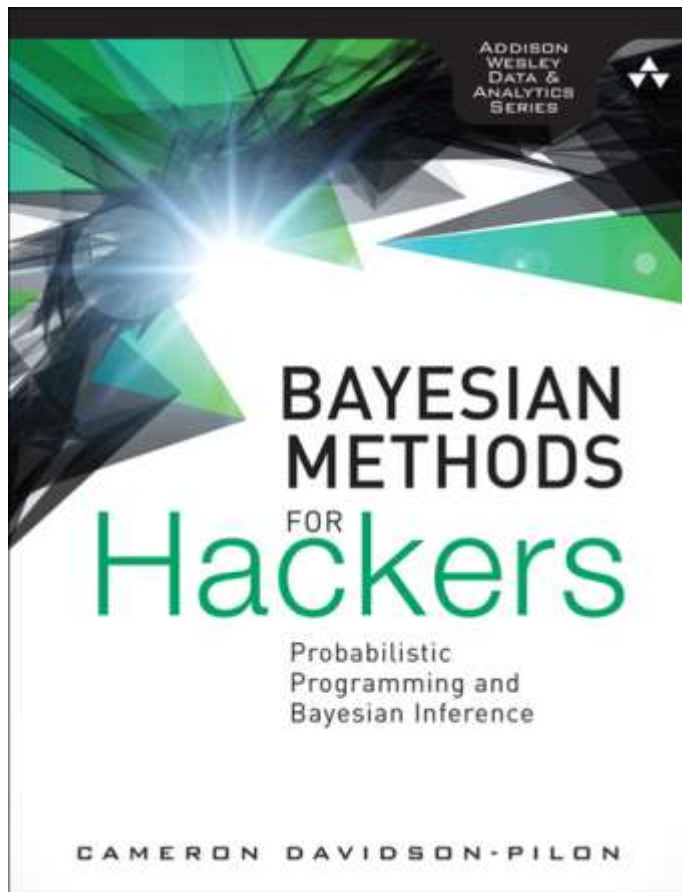
Jan-Willem van de Meent
College of Computer and Information Science
Northeastern University
j.vandemeent@northeastern.edu

Brooks Paige
Alan Turing Institute
University of Cambridge
bpaige@turing.ac.uk

Hongseok Yang
School of Computing
KAIST
hongseok.yang@kaist.ac.kr

Frank Wood
Department of Computer Science
University of British Columbia
fwood@cs.ubc.ca





<https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>

GitHub - CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers

.github	Create FUNDING.yml	last year
Chapter1_Introduction	ch1 little modify	last year
Chapter2_MorePyMC	corrected all the occurrences of pytensor import	8 months ago
Chapter3_MCMC	corrected pytensor import	8 months ago
Chapter4_TheGreatest...	all review problems solved	8 months ago
Chapter5_LossFunctio...	corrected all the occurrences of pytensor import	8 months ago
Chapter6_Priorities	corrected all the occurrences of pytensor import	8 months ago
Chapter7_BayesianMa...	docs: Fix a few typos	2 years ago
ExamplesFromChapters	Fix typos in code examples.	7 years ago
Prologue	Fix a typo in the prologue.	6 years ago
sandbox	normed changed to density	2 years ago
styles	Fixed #395	3 years ago
.gitignore	Do not track IPython notebook checkpoints	9 years ago
LICENSE.txt	Update LICENSE.txt	10 years ago
README.md	Fix markdown headings	6 years ago
book_layout.txt	changed BMH_layout to book_layout, made ch...	11 years ago
requirements.txt	Update dependency.	8 months ago
to_latex_pdf.sh	Fix Python3 notebooks latex syntax	7 years ago

Bayesian methods + probabilistic programming with a computation/understanding-first, mathematics-second point of view. All in pure Python ;)

camdavidsonpilon.github.io/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers/

data-science statistics jupyter-notebook bayesian-methods pymc mathematical-analysis

Readme MIT license Activity 25.9k stars 1.4k watching 7.8k forks Report repository

Releases No releases published

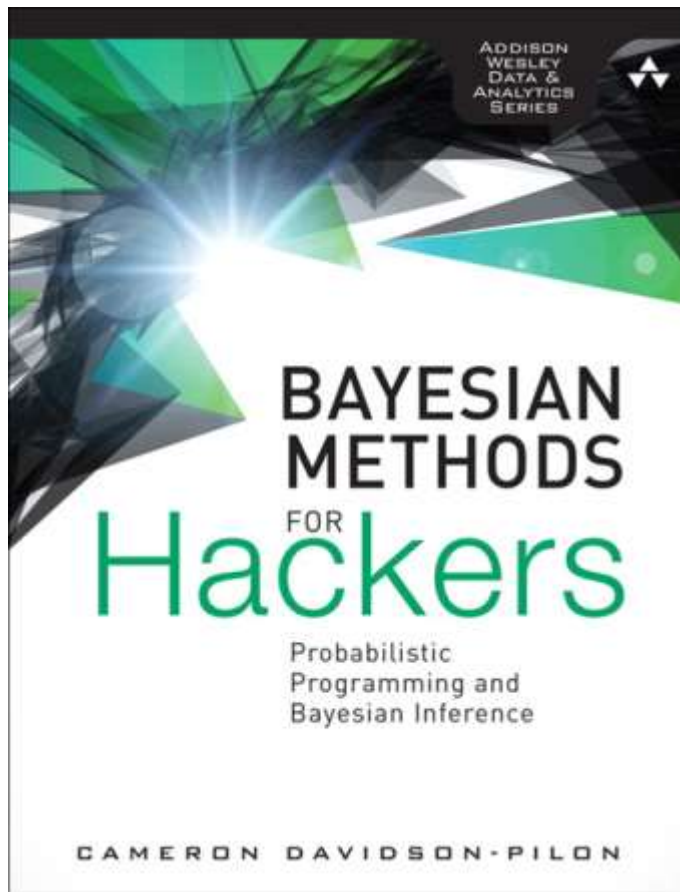
Sponsor this project

CamDavidsonPilon Cameron D...

Sponsor

Learn more about GitHub Sponsors

Bayesian Methods for Hackers



<https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>

Probabilistic-Programming-and

github.com/CamDavidsonPilon/Probabilistic-Progra...

<> Code Issues 153 Pull requests 47 Actions Projects Wiki

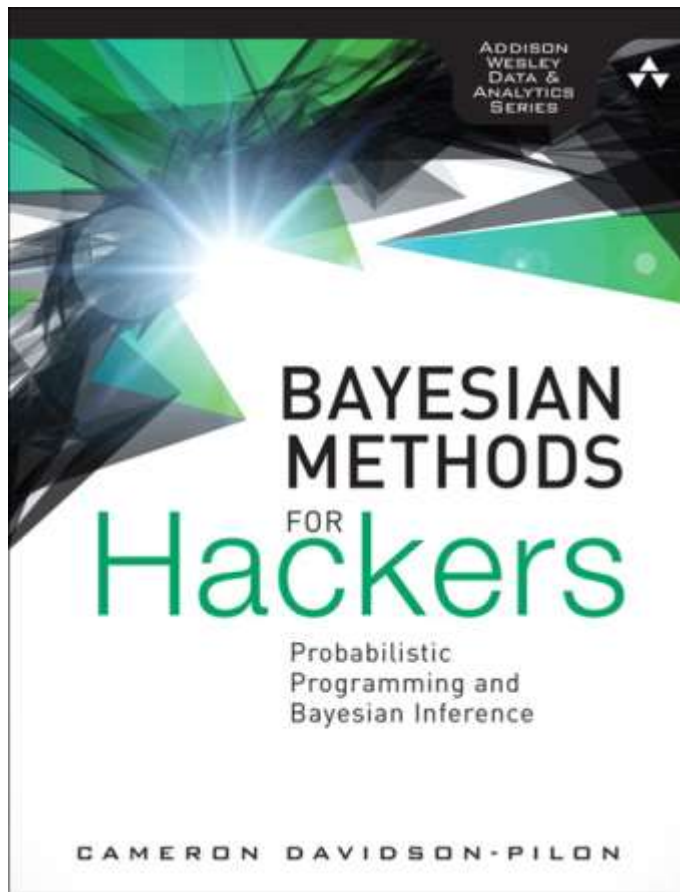
master Probabilistic-Programming-and-Bayesian-Methods-for-Hackers / Chapter1_Introduction /

miemiekurisu ch1 little modify adee836 · last year History

Name	Last commit message	Last commit date
..		
data	cleaning up some folders and changing the c...	10 years ago
Ch1_Introduction_PyMC2.ipynb	normed changed to density	2 years ago
Ch1_Introduction_PyMC3.ipynb	Remove unused theano import	2 years ago
Ch1_Introduction_PyMC_current.ipynb	ch1 little modify	last year
Ch1_Introduction_Pyro.ipynb	Adding port to Pyro - Chapter 1	3 years ago
Ch1_Introduction_TFP.ipynb	Update to TF2, plus other refactorings	3 years ago
README.md	Fix chapter 1-6 read-online links	7 years ago

README.md

Chapter 1: Introduction to Bayesian Methods



<https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>

Probabilistic-Programming-and-...
github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers/blob/master/Chapter2_Mor...
master
Probabilistic-Programming-and-Bayesian-Methods-for-Hackers / Chapter2_MorePyMC
/ Ch2_MorePyMC_PyMC_current.ipynb
1.56 MB
Download

Original content created by Cam Davidson-Pilon

Ported to Python 3 and PyMC3 by Max Margenot (@clean_utensils) and Thomas Wiecki (@twiecki) at Quantopian (@quantopian)

Ported to PyMC last by Kurisu Chan (@miemiekurisu)

This chapter introduces more PyMC syntax and variables and ways to think about how to model a system from a Bayesian perspective. It also contains tips and data visualization techniques for assessing goodness-of-fit for your Bayesian model.

A little more on PyMC

Model Context

In PyMC, we typically handle all the variables we want in our model within the context of the `Model` object.

```
In [1]: import pymc as pm
import pytensor
import numpy as np
RANDOM_SEED = 8927
rng = np.random.default_rng(RANDOM_SEED)
# %config InlineBackend.figure_format = 'retina'
# the proper way to init the value is to use initval parameter
# The test_value is just for debug
# reference is at https://pytensor.readthedocs.io/en/latest/tutorial/debug_faq.html#using-test-values
# and https://github.com/pymc-devs/pymc/issues/562#issuecomment-932146862
# You can use pytensor.config.compute_test_value = 'warn' to debug
with pm.Model() as model:
    parameter = pm.Exponential("poisson_param", 1.0, initval=rng.exponential(1))
    data_generator = pm.Poisson("data_generator", parameter)
```

This is an extra layer of convenience compared to PyMC. Any variables created within a given `Model`'s context will be automatically assigned to that model. If you try to define a variable outside of the context of a model, you will get an error.

“Hello World!”

Example: Coin Tossing

- We have a coin which may be fair or not
- The task is to estimate a probability θ of landing heads up
- Data: n tries with a result of x heads

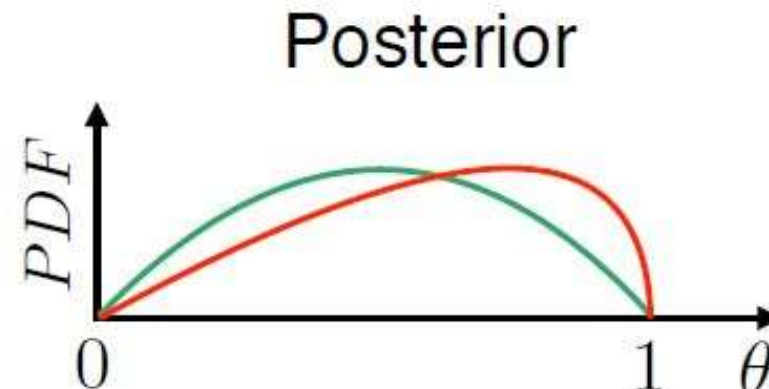
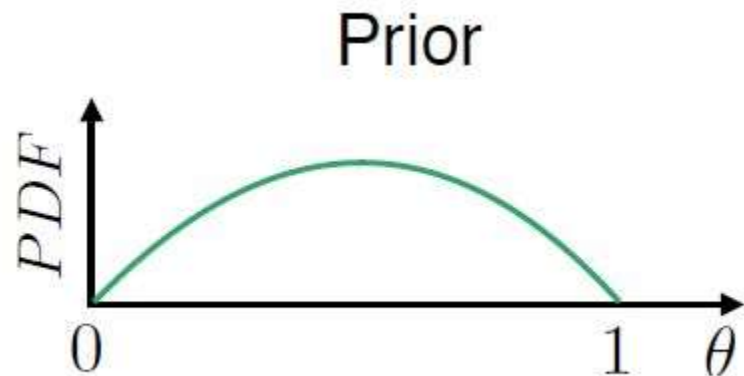


Head (H)

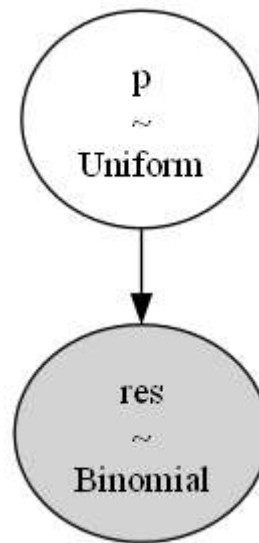


Tail (T)

Bayesian framework:



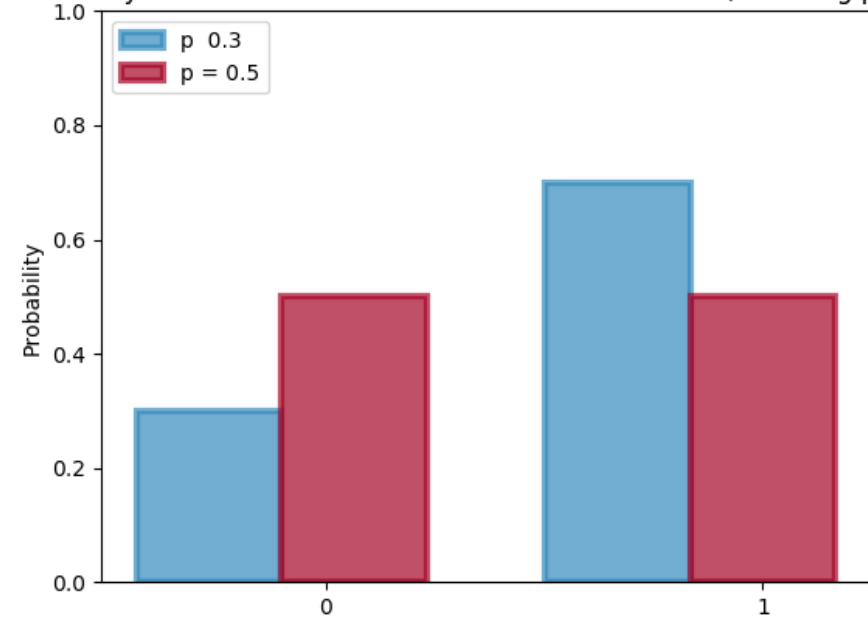
The Model



Bernoulli Distribution

$$Z \sim \text{Ber}(p)$$
$$P(Z = k) = p^k(1 - p)^{1-k}, \quad k = 0, 1$$
$$E(Z|p) = \sum_{k=0}^1 kP(Z = k) = p$$

Probability mass function of a Bernoulli random variable; differing p values

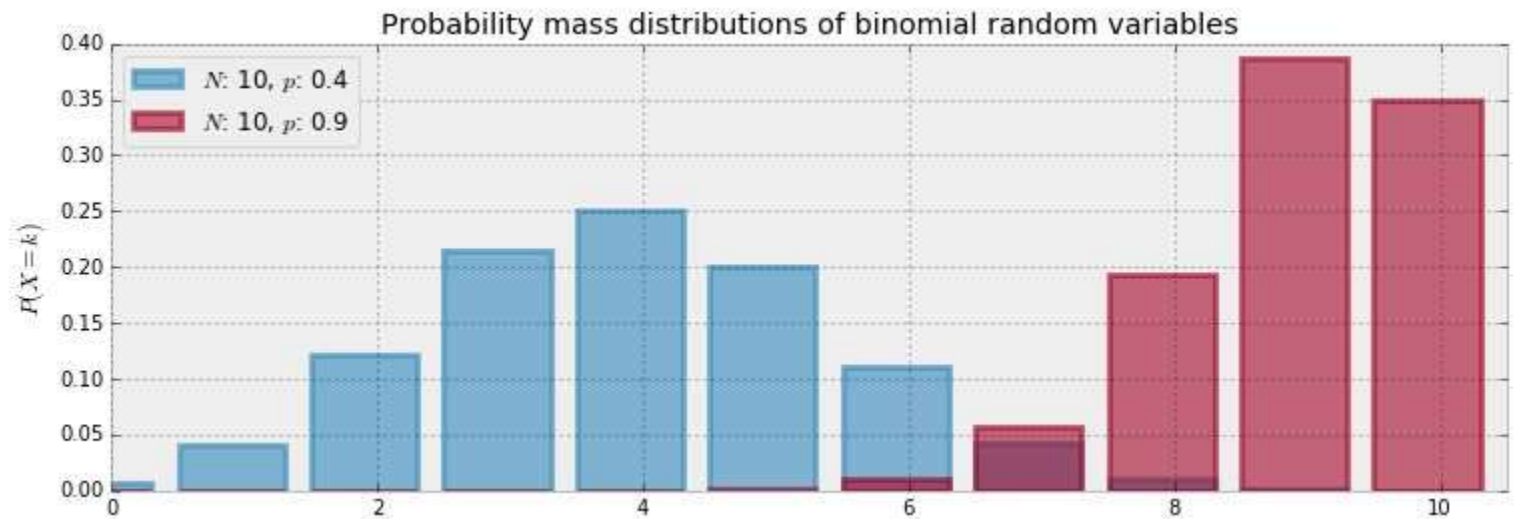


```
Z = pymc.Bernoulli("Z", p)
```

Binomial Distribution

$$Z \sim \text{Bin}(N, p)$$
$$P(Z = k) = \binom{N}{k} p^k (1 - p)^{N-k}, \quad k = 0, 1, \dots, N$$
$$E(Z|N, p) = \sum_{k=0}^N k P(Z = k) = Np$$

Z is the number of events that occurred in the N trials, and p is the probability of a single event



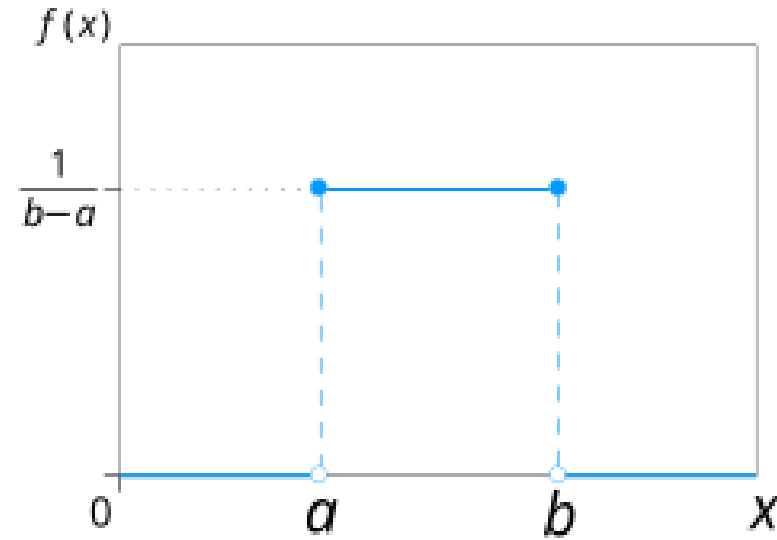
```
Z = pymc.Binomial("Z", N, p)
```

Continuous Uniform Distribution

$$Z \sim U_{(a,b)}$$

$$f_Z(z|a,b) = \begin{cases} \frac{1}{b-a} & \text{for } z \in [a,b] \\ 0 & \text{otherwise} \end{cases} \quad -\infty < a < b < \infty$$

$$E(Z|a,b) = \frac{1}{2}(a+b)$$



```
Z=pymc.Uniform("Z", a, b)
```

PyMC Solution

```
import pymc as pm
import graphviz as gv
import numpy as np
import matplotlib.pyplot as plt

true_head_p = 0.7
n = 100
obs = np.random.binomial(n, true_head_p)

with pm.Model() as coin_model:
    p = pm.Uniform('p')
    res = pm.Binomial('res', n, p, observed=obs)

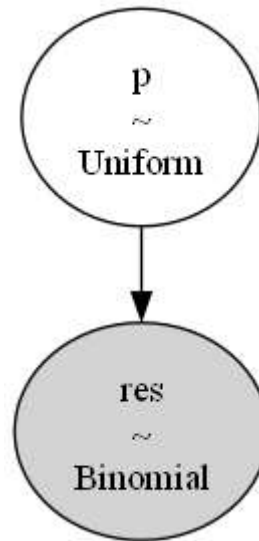
gv = pm.model_to_graphviz(coin_model)
gv.format = 'png'
gv.render(filename = 'model_graph')

### Mysterious code to be explained in Chapter 3.
with coin_model:
    step = pm.Metropolis()
    trace = pm.sample(40000, tune=10000, chains=1, step=step, return_inferencedata=False)

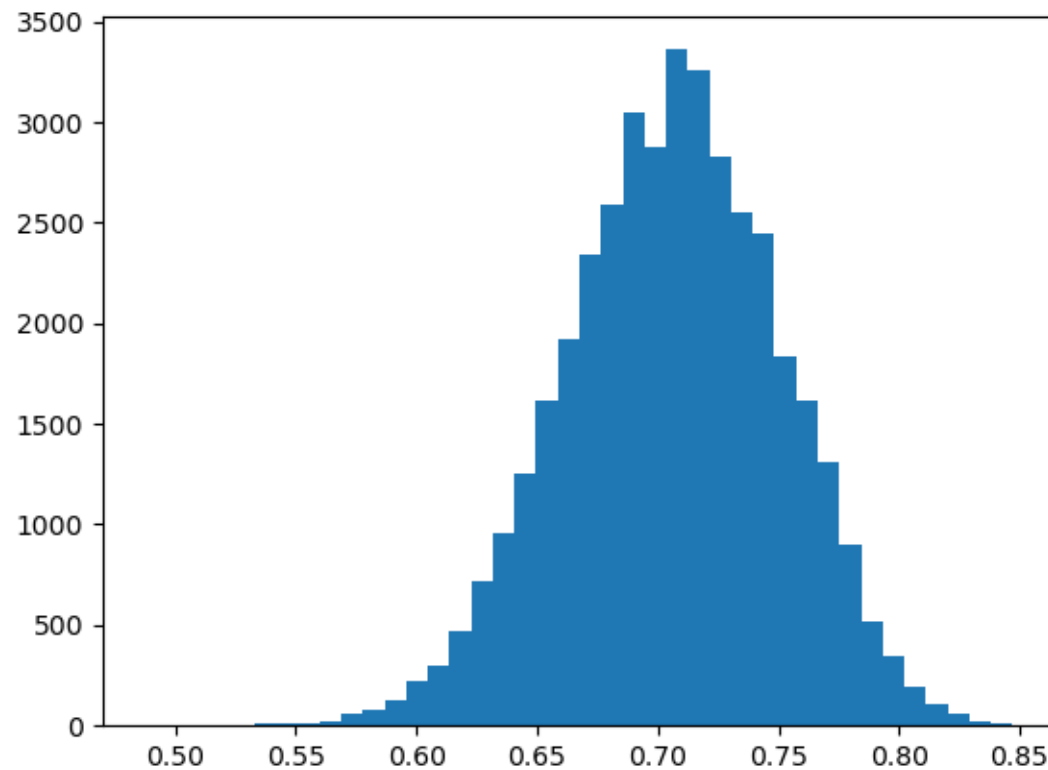
p_samples = trace['p']

plt.hist(p_samples, bins = 40)
plt.show()
```

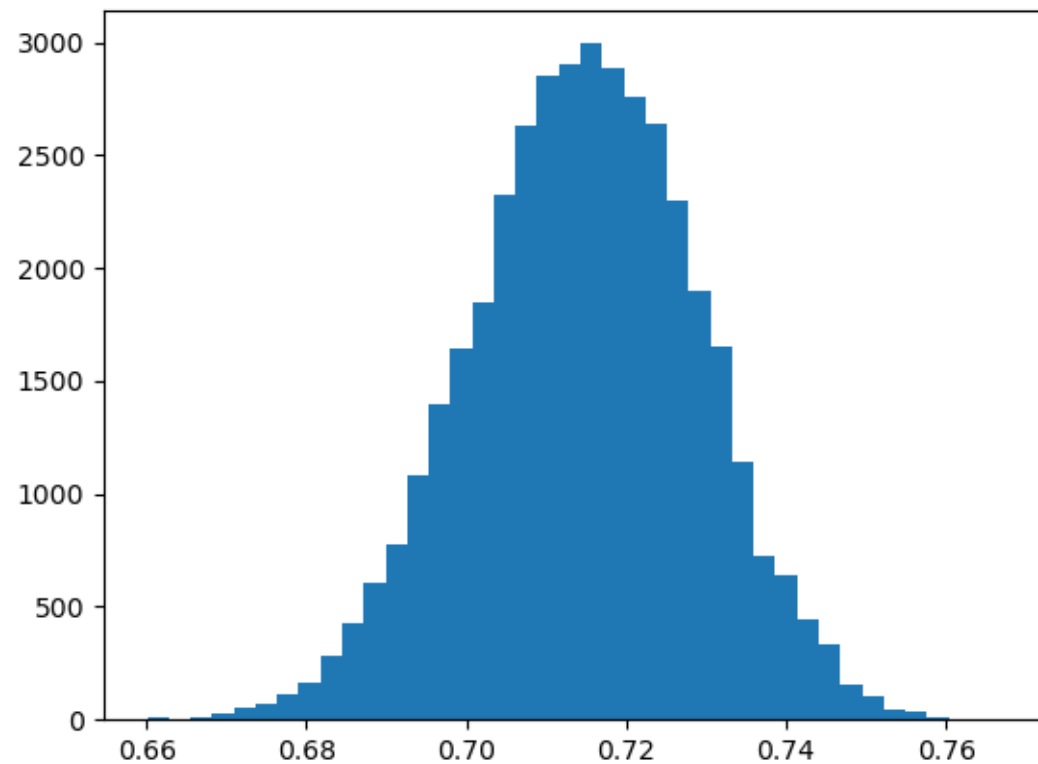

The Model (Rendered by Graphviz)



Result: Posterior Distribution of p ($n = 100$)



Result: Posterior Distribution of p ($n = 1000$)



Administrative Details

One Project Assignment

	Release date	Due date
Project	Week 9	Week 15

- The assignment will be graded from 0 to 10
- Collaboration: All students must work individually

Grading

- Project – 70%
- Exam – 30%